

# Synthesizable Matrix-Vector Multiplier Design and Analysis

---

**Dhirajzen Bagawath Geetha Kumaravel**

NetID: db5309

Advanced Projects — ECE-GY 9953

New York University — Tandon School of Engineering

Target PDK: TSMC 16 nm

---

## 1. Introduction and Motivation

This project originated from a directed study of the **TSMC 16 nm PDK**, specifically the compiled-SRAM Verilog models delivered by the TSMC memory compiler. Those models expose both a behavioral timing description and a structural gate-level netlist covering the sense-amplifier, row-decoder, write-driver, and column-mux cells instantiated in each SRAM bank. Examining the timing arcs, power-state annotations, and read-port bandwidth figures in those files surfaced a recurring observation: in weight-stationary neural-network inference pipelines, the dominant cycle-time constraint is not the multiply-accumulate (MAC) arithmetic itself but the rate at which weight data can be streamed from the on-chip SRAM bank into the compute fabric. Even a modest 32x32 weight tile requires  $32 \times 8 = 256$  bits of SRAM read bandwidth per inference step, and multi-bank arbitration or ECC overhead can further reduce effective throughput below the raw port width.

Correctly sizing the memory interface therefore requires knowing the exact compute latency that the arithmetic datapath can sustain. This motivated designing a fully characterised, synthesizable matrix-vector multiplier (MVM) targeting the TSMC 16 nm standard-cell process, so that the SRAM front-end can be dimensioned around measured RTL timing rather than rough estimates. The TSMC 16 nm FinFET node is particularly well-suited to this class of design: its three-sided gate geometry suppresses short-channel effects and drain-induced barrier lowering (DIBL), raising drive current and reducing the FO4 unit delay to approximately **11 ps** at the nominal SVT corner — roughly 30% faster than the planar TSMC 28 nm node (~15 ps). The target compute operation is  $\mathbf{y} = \mathbf{W} \cdot \mathbf{x}$ , where  $\mathbf{W}$  is a 32x32 matrix of signed 8-bit weights,  $\mathbf{x}$  is a 32-element vector of signed 2-bit activations drawn from the set  $\{-2, -1, 0, +1\}$  (consistent with 2-bit post-training quantization schemes), and  $\mathbf{y}$  is a 32-element vector of signed 15-bit outputs. The 2-bit activation constraint is intentional: it collapses the general multiplier into a trivial decode network, as detailed in Section 3.

## 2. Design Specification and Bit-Width Derivation

Every numeric width in the design is derived analytically from worst-case signal bounds so that no overflow can occur at any intermediate node without requiring saturation logic or run-time checks. Table 1 summarises the full parameter set; the derivations follow.

| Parameter                 | Symbol | Value            | Derivation / Justification                                                       |
|---------------------------|--------|------------------|----------------------------------------------------------------------------------|
| Matrix / vector dimension | N      | 32               | Tile size; matches common DNN layer widths                                       |
| Input element width       | IN_W   | 2 bits (signed)  | Activations in $\{-2, -1, 0, +1\}$ ; 2-bit quantization                          |
| Weight element width      | W_W    | 8 bits (signed)  | INT8 weights; range $\{-128, +127\}$                                             |
| Product width             | PROD_W | 10 bits (signed) | $\text{IN\_W} + \text{W\_W}$ ; max $ \text{product}  = 2 \times 128 = 256 < 2^9$ |
| Adder-tree depth          | LEVELS | 5                | $\text{clog}_2(32) = 5$ reduction levels                                         |
| Accumulator width         | ACC_W  | 15 bits (signed) | $\text{PROD\_W} + \text{LEVELS}$ ; max $ \text{sum}  = 8192 = 2^{13} < 2^{14}$   |

Table 1. Module parameters and bit-width derivation.

**Product width (PROD\_W = 10).** The signed 2-bit input takes values  $\{-2, -1, 0, +1\}$  and the signed 8-bit weight spans  $\{-128, +127\}$ . The extremal products are  $(-2) \times (-128) = +256$  and  $(-2) \times (+127) = -254$ . Representing +256 in two's complement requires 9 magnitude bits plus one sign bit, i.e., exactly 10 bits. The general formula  $\text{PROD\_W} = \text{IN\_W} + \text{W\_W}$  gives this bound tightly.

**Accumulator width (ACC\_W = 15).** Summing  $N = 32$  products each bounded by  $|p| \leq 256$  gives a maximum output magnitude of  $32 \times 256 = 8192 = 2^{13}$ . A 15-bit signed register covers  $[-16384, +16383]$ , so the result always fits with two bits of headroom. The formula  $\text{ACC\_W} = \text{PROD\_W} + \text{clog}_2(N) = 10 + 5 = 15$  captures this exactly: each level of the adder tree requires one additional guard bit to prevent overflow when two equal-magnitude numbers are added.

## 3. RTL Architecture

The Verilog module **matvec\_mul** is fully combinational and parameterised via *localparam* expressions that derive PROD\_W, LEVELS, and ACC\_W automatically from N, IN\_W, and W\_W. All ports are packed bit-vectors so the module remains synthesizable on any standard-cell flow without multi-dimensional port declarations. A single outer *generate-for* loop over index  $i$  (0 to N-1) replicates 32 identical dot-product engines, one per output element  $y[i]$ , each consisting of two pipelined sub-stages.

### 3.1 Stage A — Multiplier Array

Each dot-product engine instantiates an inner generate loop over  $j$  (0 to  $N-1$ ), producing 32 signed multiply cells for a total of **1024 cells** across the design. Before each multiplication, both operands are explicitly sign-extended to  $PROD\_W = 10$  bits. This is critical for correctness: if a 2-bit signed operand is multiplied directly against an 8-bit signed operand in SystemVerilog without prior extension, the tool extends both to the larger operand width (8 bits), which cannot represent the product +256 and silently overflows. Explicit sign extension to 10 bits before the multiply avoids this. Because  $IN\_W = 2$ , only four input values are possible, and Design Compiler reduces each multiply cell to a 4-way static decode (Table 2). No Wallace or Dadda partial-product reduction tree is generated.

| vec_in[j] (2-bit signed) | Value | Hardware realisation on W[7:0]                                    |
|--------------------------|-------|-------------------------------------------------------------------|
| 2'b00                    | 0     | Product = 0; output permanently tied to GND — zero cells consumed |
| 2'b01                    | +1    | Product = W; sign-extend to 10 bits — wiring only, no logic       |
| 2'b11                    | -1    | Product = -W; bit-invert W[9:0] and assert carry-in = 1           |
| 2'b10                    | -2    | Product = -2W; left-shift W by 1 (wiring), then negate as above   |

Table 2. Multiply-cell realisation for each 2-bit signed input value.

The left-shift by 1 in the -2 case is free: it is implemented by routing weight bit[ $i$ ] to output wire bit[ $i+1$ ], with bit[0] tied to GND — no gate is instantiated. The negate step (invert + carry-in) costs approximately 10 XOR gates plus a short carry chain. Overall each cell synthesises to **~20-30 NAND2-equivalent gates**, compared to ~80-120 for a general 2x8 signed multiplier — a 3-5x area and timing reduction. Sign-extension of the 10-bit product to  $ACC\_W = 15$  bits is pure wiring (the sign bit is fanned out to bits [14:10]) and adds zero cells.

### 3.2 Stage B — Five-Level Binary Adder Tree

A linear accumulator chain would place all 31 adders on the critical path. A balanced binary tree reduces this to  $\log_2(32) = 5$  adder levels while using the same number of adders, trading latency for throughput. All intermediate wires are kept at  $ACC\_W = 15$  bits throughout; the accumulation range [-8192, +8192] cannot overflow this width at any level. Synopsys Design Compiler maps each 15-bit addition to a carry-lookahead (CLA) or carry-select (CSA) structure from the TSMC 16 nm SVT library, achieving approximately 4-5 logic levels per adder. Per dot-product engine: 31 adders x 15 bits = **465 FA cells**. Across all 32 rows: 992 adders, approximately **14,880 FA cells** total.

| Tree Level | Inputs   | Outputs  | Adders per Row | Cumulative |
|------------|----------|----------|----------------|------------|
| 0 -> 1     | 32 x 15b | 16 x 15b | 16             | 16         |
| 1 -> 2     | 16 x 15b | 8 x 15b  | 8              | 24         |
| 2 -> 3     | 8 x 15b  | 4 x 15b  | 4              | 28         |
| 3 -> 4     | 4 x 15b  | 2 x 15b  | 2              | 30         |
| 4 -> 5     | 2 x 15b  | 1 x 15b  | 1              | 31         |

Table 3. Binary adder tree reduction schedule per dot-product engine.

## 4. Timing Analysis — TSMC 16 nm

### 4.1 Fan-Out-of-4 Delay Metric

All delays are expressed in **fan-out-of-4 (FO4)** units — the propagation delay of a minimum-sized inverter driving four identical inverters. FO4 is technology-independent by construction: it scales with the node's intrinsic gate delay and is routinely used by Synopsys and Cadence to compare designs across process generations without committing to a specific library. For the TSMC 16 nm SVT library at the nominal operating corner ( $V_{DD} = 0.8$  V, TT process, 25 C), published characterisation data places **FO4  $\approx$  11 ps**. This is used as the base unit for all delay estimates below.

### 4.2 Critical-Path Breakdown

The combinational critical path for any single output element  $y[i]$  traverses four segments in sequence. The **multiplier decode** stage contributes  $\sim 4$  FO4: one level for the 2-bit input decode ( $\sim 1$  FO4), one for the XOR-based conditional invert of the weight bits ( $\sim 1$  FO4), and  $\sim 2$  FO4 for the carry propagation through the 10-bit conditional incrementer. The **sign extension** to 15 bits is pure wiring and adds zero delay. Each **15-bit CLA adder** in the tree contributes  $\sim 5$  FO4: the carry lookahead logic resolves in  $\sim 3$  FO4 across two grouping levels, and the final sum XOR adds  $\sim 2$  FO4. With 5 tree levels this gives 25 FO4. A **routing and clock margin** of  $\sim 5$  FO4 is added to account for wire RC parasitics not captured at gate level, OCV (on-chip variation), and clock uncertainty. Table 4 shows the full breakdown.

| Critical-Path Segment                  | FO4 Count     | Delay at 11 ps/FO4 | Share of Path |
|----------------------------------------|---------------|--------------------|---------------|
| Multiplier decode + conditional negate | $\sim 4$ FO4  | $\sim 44$ ps       | 12%           |
| Sign extension to 15 bits (wiring)     | 0             | 0 ps               | 0%            |
| Adder tree Level 1 — 15-bit CLA        | $\sim 5$ FO4  | $\sim 55$ ps       | 15%           |
| Adder tree Levels 2-5 — 4 x 15-bit CLA | $\sim 20$ FO4 | $\sim 220$ ps      | 59%           |
| Routing parasitics + clock margin      | $\sim 5$ FO4  | $\sim 55$ ps       | 15%           |
| Total combinational critical path      | $\sim 34$ FO4 | $\sim 374$ ps      | 100%          |

Table 4. Critical-path breakdown at TSMC 16 nm nominal corner (TT, 0.8 V, 25 C).

The adder tree accounts for **74% of the total delay** (25 of 34 FO4). Within the tree, Level 1 is the dominant bottleneck: its 16 adders per row receive multiplier outputs directly, where bit-entropy is at its maximum and carry-chain probability is highest. Levels 2-5 see progressively more stable partial sums, but because they still traverse CLA logic their per-level delay is identical — each contributes  $\sim 5$  FO4.

### 4.3 Post-Route Estimate and Maximum Frequency

Gate-level FO4 analysis captures only intrinsic cell delay. After place-and-route, wire resistance and inter-metal coupling capacitance lengthen each timing arc. A standard industry derating factor of **1.5x** is applied to convert the gate-level path to a post-layout estimate, consistent with TSMC 16 nm routing density and typical datapath floorplan utilisation ( $\sim 70\%$ ).

$$T_{crit}(\text{post-route}) = 374 \text{ ps} \times 1.5 = 561 \text{ ps} \rightarrow F_{max} = 1/561 \text{ ps} \approx 1.78 \text{ GHz}$$

Subtracting a 10% guard-band for clock tree uncertainty and setup-time margin gives a practical operating target of **1.5 – 1.6 GHz** at the nominal corner.

### 4.4 PVT Corner Analysis

Process, voltage, and temperature (PVT) variation shifts the FO4 unit delay and therefore the entire critical path. The slow corner (SS process, 125 C, 0.72 V) represents the worst-case timing scenario: slower transistors, lower overdrive voltage, and higher lattice scattering increase FO4 to  $\sim 17$  ps. The fast corner (FF process, -40 C, 0.88 V) gives the best-case timing with FO4  $\sim 8$  ps. Table 5 tabulates all three corners.

| PVT Corner         | V <sub>DD</sub> | FO4   | Logic Path (34 FO4) | Post-Route (1.5x) | Practical F <sub>max</sub> |
|--------------------|-----------------|-------|---------------------|-------------------|----------------------------|
| TT, 25 C — nominal | 0.8 V           | 11 ps | 374 ps              | 561 ps            | 1.5 – 1.8 GHz              |
| SS, 125 C — slow   | 0.72 V          | 17 ps | 578 ps              | 867 ps            | 900 MHz – 1.1 GHz          |
| FF, -40 C — fast   | 0.88 V          | 8 ps  | 272 ps              | 408 ps            | 2.0 – 2.4 GHz              |

Table 5. Timing estimates across PVT corners, TSMC 16 nm SVT library.

### 4.5 Pipelining and Synthesis Strategy

The combinational design is sign-off ready for single-cycle operation up to ~1.6 GHz. For higher-throughput targets, a single pipeline register bank inserted between adder Levels 2 and 3 splits the 34 FO4 path into two balanced halves of ~19 FO4 each (~209 ps logic, ~315 ps post-route). This more than doubles achievable frequency to **>3 GHz at TT**, at the cost of  $32 \text{ rows} \times 15 \text{ bits} = \mathbf{480 \text{ D flip-flops}}$  and a 2-cycle compute latency. For a throughput-bound inference engine this trade is almost always beneficial.

Under Synopsys Design Compiler *compile\_ultra*, the recommended synthesis constraints for the combinational version are:

| DC Constraint         | Value              | Purpose                                                              |
|-----------------------|--------------------|----------------------------------------------------------------------|
| set_max_delay         | 560 ps (data path) | Enforce post-route timing budget on all combinational paths          |
| set_max_fanout        | 8                  | Prevent high-fanout nets from adding buffer delay on packed buses    |
| set_max_transition    | 120 ps             | Limit slew on long wires; keeps cell characterisation valid          |
| compile_ultra -retime | enabled            | Allows DC to move logic across virtual registers during optimisation |

Table 6. Recommended Synopsys DC synthesis constraints.

## 5. Conclusion

A synthesizable, fully combinational 32x32 signed matrix-vector multiplier has been designed and characterised for the TSMC 16 nm standard-cell process. Exploiting the 2-bit activation constraint eliminates Wallace-tree multipliers entirely, replacing each cell with a 4-way decode-and-negate network of ~20-30 NAND2-equivalent gates — a 3-5x area reduction over a general signed multiplier. Sign extension from the 10-bit product to the 15-bit accumulator width is resolved in wiring with zero cell cost. A five-level balanced binary adder tree reduces 32 products per output row to a single 15-bit result through 31 carry-lookahead adders with no intermediate overflow at any tree level. Timing analysis at the TSMC 16 nm nominal corner (TT, 0.8 V, 25 C) yields a gate-level critical path of ~374 ps (34 FO4) and a post-route estimate of ~560 ps, giving **F\_max of 1.5-1.8 GHz** for the single-cycle combinational version. Inserting one pipeline register bank at mid-tree raises this above 3 GHz with only 480 flip-flops of overhead. The TSMC SRAM study that motivated this project confirmed that weight-matrix read bandwidth — not multiply-accumulate compute — is the binding bottleneck in inference pipelines. A characterised MVM datapath with known critical-path delay is therefore the necessary foundation for correctly sizing the accompanying SRAM interface and memory controller.

## References

- [1] TSMC 16 nm Process Design Kit, Compiled SRAM Memory Compiler Reference Manual, Rev. 3.0.
- [2] TSMC 16 nm FinFET Standard-Cell Library Databook, SVT/HVT/LVT Characterisation Report.
- [3] Synopsys Design Compiler User Guide — compile\_ultra Strategy, Version S-2021.06.
- [4] M. Rastegari et al., "XNOR-Net: ImageNet Classification Using Binary CNNs," *ECCV*, 2016.
- [5] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design*, 4th ed. Addison-Wesley, 2011.
- [6] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits*, 2nd ed. Prentice Hall, 2003.